

IT549: Deep Learning

Lab Assignment 4: Object Detection Evolution: From R-CNN to YOLO

Dataset: [fruit images for object detection](#)

Objective: This lab introduces students to the evolution and practical implementation of object detection models. Students will explore the architectural differences between early region-proposal models (R-CNN, Fast R-CNN) and implement inference and fine-tuning using modern architectures (Faster R-CNN and YOLO).

Dataset Scope: For this assignment, you will use the **Fruit Images dataset** available on Kaggle.

- It contains 3 classes (Apple, Banana, Orange).
 - Bounding box labels are provided in both normalized (x_center , y_center , width, height) and (x_min , y_min , x_max , y_max) formats.
-

Tasks

Preparation: Ground Truth Visualization

- Load a single, random image from your downloaded dataset.
- Parse the corresponding annotation file (e.g., .txt file or .xml file) to extract the ground truth bounding box coordinates and class labels.
- **Visualize:** Use Matplotlib and OpenCV to plot the image and draw all ground-truth bounding boxes on top, including text labels for each box.

Task 1 - Core Concept: Intersection over Union (IoU)

- Implement a Python function from scratch to calculate the Intersection over Union (IoU) between two bounding boxes.
- The function should accept two boxes in $[x_min, y_min, x_max, y_max]$ format and return a float representing the IoU score.
- **Demonstrate:** Show the function working by passing at least three hardcoded pairs of boxes (e.g., one highly overlapping pair, one partially overlapping pair, and one completely disjoint pair) and printing the resulting scores.

Task 2 - The Baseline: Selective Search (R-CNN Step 1)

- Load a random image from your dataset.
- Implement OpenCV's `SelectiveSearchSegmentation`.
- Extract the first 200 region proposals (bounding boxes).

- **Visualize:** Plot the original image and overlay the 200 bounding boxes using Matplotlib.

Task 3 - Implementing the R-CNN Bottleneck. In R-CNN, every single region proposal is cropped and passed through a CNN independently.

- Load a pre-trained PyTorch classification model (e.g., `torchvision.models.resnet18`). Remove the final fully connected classification layer, so it outputs feature vectors.
- Write a loop that:
 - Iterates through the top 100 bounding boxes from Task 2.
 - Crops that exact region from the original image tensor.
 - Resizes the crop to 224x224 (the required input size for ResNet).
 - Passes the resized crop through the ResNet model to extract its features.
- **Analysis:** Record exactly how long it takes to process these 100 crops. Print the total execution time. (You can use Python's `time` module.)

Task 4 - Implementing Fast R-CNN (RoI Pooling) Fast R-CNN solved the bottleneck from Task 3 by passing the entire image through the CNN only once, and then pooling features directly from the resulting feature map.

- Pass the entire, uncropped original image through the ResNet18 convolutional layers to generate a single feature map.
- Calculate the spatial scale (stride) of your feature map (e.g., if a 224x224 image produces a 7x7 feature map, the spatial scale is 1/32).
- Use PyTorch's built-in `torchvision.ops.roi_pool`. Pass it:
 - The single feature map you just generated.
 - The same 100 bounding boxes from Task 2 (converted to tensor format).
 - An output size of the feature map and the spatial scale calculated above.
- **Analysis:** Time this entire process (1 CNN pass + 1 RoI pool). Compare this execution time to the time recorded in Task 3.
- **Conceptual Analysis:** In a Markdown cell, answer the following: *Based on your recorded execution times, you should observe a massive speedup when transitioning from the R-CNN approach to the Fast R-CNN approach. What redundant mathematical operations does Fast R-CNN eliminate to achieve this efficiency?*

Task 5 - Faster R-CNN Faster R-CNN replaces Selective Search with an internal Region Proposal Network (RPN).

- Load a complete, pre-trained `faster_rcnn_resnet50_fpn` from `torchvision`.
- Pass the same test image through the model.
- Extract the predicted bounding boxes, labels, and confidence scores.
- Write a function to filter out predictions with a confidence score below 0.80.
- **Visualize:** Plot the image with the final, filtered bounding boxes drawn on top.

- **Conceptual Analysis:** In a Markdown cell, answer the following: *Faster R-CNN introduced the Region Proposal Network (RPN). How does the RPN eliminate the need for external algorithms like Selective Search?*

Task 6 - Non-Maximum Suppression (NMS) Modern object detectors often predict multiple overlapping bounding boxes for the exact same object. NMS is the algorithm used to keep only the most accurate box and discard the redundant ones.

- **Implement the NMS Algorithm:** Write a custom NMS function that takes a list of bounding boxes, their corresponding confidence scores, and an IoU threshold (e.g., 0.5). Using the **IoU function you created in Task 1**, your algorithm must:
 - Sort the boxes by confidence score in descending order.
 - Select the box with the highest score and save it as a "final prediction."
 - Compare this selected box to all remaining boxes using your IoU function.
 - Discard any remaining boxes with IoU greater than the threshold relative to the selected box.
 - Repeat this process for the next highest-scoring box until no boxes remain unchecked.
- **Conceptual Analysis:** In a Markdown cell, answer the following: *The IoU threshold acts as a strictness dial for discarding duplicate bounding boxes. If you are trying to detect a cluster of tightly packed objects (e.g., several apples piled together in a basket), how would setting the IoU threshold extremely high (0.9) versus extremely low (0.1) alter your final visual predictions? Explain why.*

Task 7 - YOLO Fine-Tuning YOLO frames object detection as a single regression problem, completely removing the region proposal stage.

- Install the **ultralytics** package.
 - Load the pre-trained **yolov8n.pt** (Nano) model.
 - Write the code to fine-tune this model on your specific downloaded dataset. Train the model for at least 10 epochs.
 - **Evaluate:**
 - Run inference on 3 images from your test set using your newly fine-tuned weights. Visualize the predictions.
 - Report the **mAP@50** and **mAP@50-95** on your test set.
 - **Comparison Table:** Create a table comparing Faster R-CNN (pre-trained), YOLO(pre-trained), and YOLO (fine-tuned) based on:
 - Inference Time (ms per image)
 - Precision and Recall
-

Deliverables (GitHub Only) Your public GitHub repository must include:

- A **README** containing the assignment title, your name, and your ID, along with the project details.
- Complete runnable code covering the IoU demonstration, manual R-CNN crop loop, Fast R-CNN RoI pooling, Faster R-CNN inference, NMS algorithm, and YOLO training.
- Ensure execution times for Task 3 and Task 4 are clearly printed in the cell outputs.
- **Include the answers to all Conceptual Analysis questions in appropriately placed Markdown cells.**

To ensure individual participation and academic integrity, the complete commit and push history must be visible.